

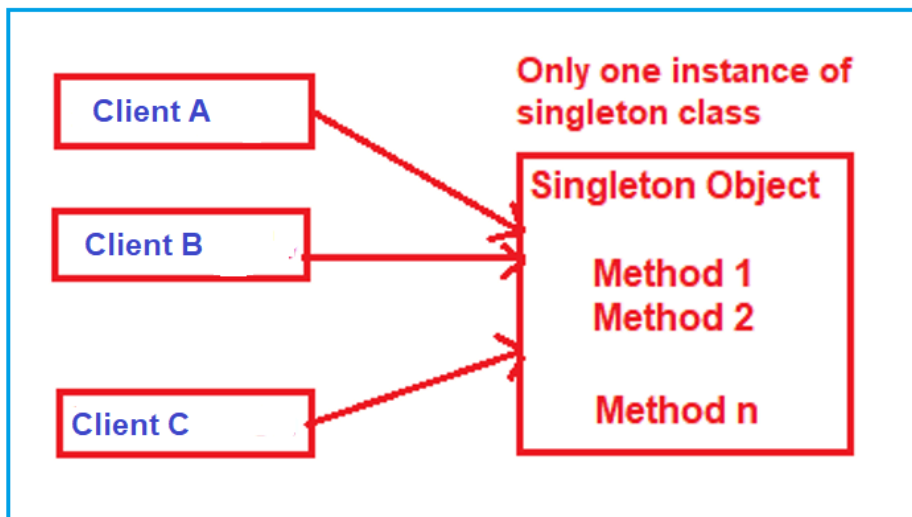
Singleton Design Pattern in C# with an Example

In this article, I will discuss the **Singleton Design Pattern in C#** with Examples. Please read our previous article discussing [Shallow Copy and Deep Copy in C#](#) with Examples. The Singleton Design Pattern in C# falls under the Creational Pattern Category. It belongs to the creational pattern category, dealing with object creation and manipulation. As part of this article, we will discuss the following pointers.

1. [What is the Singleton Design Pattern?](#)
2. [What are the Advantages of using the Singleton Design Pattern?](#)
3. [Implementation Guidelines of Singleton Pattern in C#.](#)
4. [Example of the Singleton Pattern using C#.](#)
5. [When to use Singleton Design Pattern.](#)

What is Singleton Pattern in C#?

The Singleton Design Pattern is a Creational Design Pattern used to ensure that a class has only one instance and provides a global point of access to it. In C#, the Singleton Design Pattern is useful when we need exactly one instance of a class to coordinate actions across the system. It is commonly used in scenarios where multiple objects need to access a single shared resource, such as configuration settings, access to a shared resource like a file system, or managing a connection to a database. For a better understanding, please look at the following diagram:



As you can see in the above diagram, different clients (Client A, Client B, and Client C) are trying to get the singleton instance. Once the client gets the singleton instance, they can invoke the methods (Method 1, Method 2, and Method n) using the same instance. If this is not clear at the moment, don't we will see this with multiple real-time examples.

Key Characteristics of the Singleton Design Pattern in C#

- **Single Instance:** This Design Pattern ensures that only one instance of the Singleton class is created throughout the application.
- **Global Access:** Provides a global access point to that instance.
- **Lazy Initialization:** We can lazily initialize the singleton instance, which means it is created when it is needed for the first time, not when the application starts.
- **Thread Safety:** When used in a multi-threaded application, the singleton needs to be thread-safe to prevent multiple instances from being created.

Implementation Guidelines of Singleton Design Pattern in C#:

The following are the guidelines to implement the Singleton Design Pattern in C#.

1. **Private Parameterless Constructor:** We need to create a **private and parameterless constructor**. This is required because it will restrict the class from being instantiated from outside the class; it will only instantiate from within the class.
2. **Sealed Class:** The Singleton class should be declared **sealed** to ensure it cannot be inherited. This will be useful when dealing with nested classes. In our [next article](#), we will discuss this scenario with an example.
3. **Static Variable:** We need to create a **private static variable** that holds the single instance of the class.
4. **Public Static Method or Property:** We also need to create a **public static property or method** that will return the singleton instance. This method or property first checks whether the singleton class instance has been created. If the singleton instance is created, it returns that instance; otherwise, it will create an instance and then return it. This static method or property provides the global point of access to the singleton instance and ensures that only one instance of the class is created.

Example of Singleton Design Pattern using C#

Let us understand the Singleton Design pattern in C# with Examples. We can implement the Singleton Design Pattern in C# in many ways. We will discuss all the following methods.

1. No Thread-Safe Singleton Design Pattern in C#
2. Thread-Safety Singleton Implementation using Lock.
3. Implementing Thread-Safety Singleton Design Pattern using Double-Check Locking.
4. Using Eager Loading to Implement Thread-Safety Singleton Design Pattern
5. Using Lazy<T> Generic Class to Implement Lazy Loading in Singleton Design Pattern.

No Thread-Safe Singleton Design Pattern Implementation in C#

Let us start by implementing the No Thread-Safe Singleton Design Pattern in C#. The rest of the implementations will be discussed in our upcoming articles. So, first, create a class file named **Singleton.cs**, and then copy and paste the following code. You can give the class name anything. The following code is also self-explained, so please go through the comment lines for a better understanding.

```
using System;
namespace SingletonDemo
{
    public sealed class Singleton
    {
        //This variable value will be increment by 1 each time the object
of the class is created
        private static int Counter = 0;
        //This variable is going to store the Singleton Instance
        private static Singleton Instance = null;
        //The following Static Method is going to return the Singleton
Instance
        public static Singleton GetInstance()
        {
            //If the variable instance is null, then create the Singleton
instance
            //else return the already created singleton instance
            //This version is not thread safe
            if (Instance == null)
            {
                Instance = new Singleton();
            }
            //Return the Singleton Instance
            return Instance;
        }
    }
}
```

```

        //Constructor is Private means, from outside the class we cannot
create an instance of this class
        private Singleton()
        {
            //Each Time the Constructor called, increment the Counter value
by 1
            Counter++;
            Console.WriteLine("Counter Value " + Counter.ToString());
        }
        //The following can be accesed from outside of the class by using
the Singleton Instance
        public void PrintDetails(string message)
        {
            Console.WriteLine(message);
        }
    }
}

```

Here, we created the Singleton class as **sealed**, which ensures that the class cannot be inherited from the derived classes. The class is created with a **private parameterless constructor**, ensuring that the class will not be instantiated from outside the class. But we can create instances within the same class. Again, we declared the instance variable private and initialized it with the null value, ensuring that only one class instance will be created based on the null condition. The public method, `GetInstance`, returns only one instance of the class by checking the value of the private variable instance. The public method `PrintDetails` can be invoked from outside the class through the singleton instance.

Consuming the Singleton Class

We are done with our first version of the singleton design pattern implementation, which is not thread-safe. Let us see how we can use the above Singleton class in our Main method of Program class. So, modify the Main method of the Program class as follows. As you can see in the code below, we are calling the `GetInstance()` static method of the Singleton class, which will return the Singleton instance, and then, using that Singleton instance, we are calling the `PrintDetails`. And we are doing this two times.

```

using System;
namespace SingletonDemo
{
    class Program
    {
        static void Main(string[] args)
        {
            //Call the GetInstance static method to get the Singleton
Instance
            Singleton fromTeachaer = Singleton.GetInstance();
            fromTeachaer.PrintDetails("From Teacher");
            //Call the GetInstance static method to get the Singleton
Instance
            Singleton fromStudent = Singleton.GetInstance();
            fromStudent.PrintDetails("From Student");
            Console.ReadLine();
        }
    }
}

```

Run the application, and it will give you the following output.

```
Counter Value 1
From Teacher
From Student
```

As you can see in the above output, it clearly shows that the private constructor is executed only once, even though we have called the `GetInstance` method twice and printed the counter value as 1. This proves that the singleton instance is created only once. The way we implement the singleton design pattern in the above example is not to be thread-safe. It means in a multithreaded environment, it can create multiple instances of the singleton class. We will discuss making the singleton class thread-safe in our [Thread-Safe Singleton Design Pattern](#) article.

When to Use Singleton Design Pattern in C# Real-time Applications?

The following are some of the real-time scenarios where using the Singleton Design Pattern can be beneficial:

- **Shared Resources Management:** For managing shared resources such as database connection pools or configuration data, where only one instance should manage the resource throughout the system.
- **Logging:** In scenarios where an application-wide logger instance needs to capture logs from various parts of an application, ensuring that the logging mechanism is consistently used.
- **Caching:** When you need to cache application data so it's accessible globally and maintained within a single object, ensuring consistency.
- **Controlled Access and Operations:** When you need to perform operations where exactly one instance is needed to coordinate actions across the system, for example, a logging class or managing access to a value shared across various parts of an application.

In the next article, I will discuss [Why we Need to use the Sealed Keyword in the Singleton Class](#), as we already have a private constructor. In this article, I explain the basic concepts of the **Singleton Design Pattern in C#** with Examples. I hope you understand why we need the Singleton Design Pattern in C# with Examples.